

摘要

仅凭 F3、Fz、F4 三路额区脑电，既要提取视觉响应，又要解释左右三角形刺激产生的差异，困难在于有效信号较弱、脑源与电极之间的对应关系不唯一。本文利用两组视觉认知实验的四份记录，依次建立试次质量筛选、视觉刺激正向计算和认知功能代理模型，分析三通道观测能够支持的波形特征与机制解释。

针对问题一，采用稳健黎曼伪迹拒绝方法，结合协方差距离和分频段统计量构造信号质量指数 SQI。位置提示任务配置四个低维检测器，形状提示任务配置三个检测器并放宽筛选阈值；剔除削顶及低质量试次后，进行基线校正、条件叠加平均和候选峰提取。再以二阶差分惩罚的三次 B 样条拟合 ERP，由广义交叉验证选择平滑参数。四份记录共保留 **297 个试次**，24 条条件平均曲线的拟合优度 R^2 为 0.891–1.000。阈值敏感性分析与等样本量随机抽样表明，整体波形相对稳定，但局部峰值仍受筛选强度影响。

针对问题二，将 LGN 的 ON/OFF 中继、Gabor 方向滤波、三角形空间构型编码与 Wilson–Cowan 型兴奋–抑制群体方程连接，构造五个功能源代理，并通过固定线性映射生成三通道脑电。采用有界 L-BFGS-B 搜索时间常数与抑制增益，左右条件共享幅度因子。空间位置消融后，模型左右差异的均方根幅度降至原来的 10^{-5} 量级，说明位置编码是该模型产生差异的关键。总体 ERP 的样本内相关系数为 **0.894**，归一化均方根误差为 0.358；左右差异波的相关系数仅为 -0.097 。基于实测脑电九维窗口特征的收缩线性判别分析，在留一记录验证中取得 46.6% 的平衡准确率，尚未获得稳定的跨记录判别能力。

针对问题三，直接从原始脑电提取 ERP 均值与频带对数功率，构造视觉前馈、记忆匹配和认知控制三类功能代理 $V/H/P$ ，以普通最小二乘描述试次内关联，并通过留一记录验证、代理消融及 Logistic 应答侧预测检验模型。按提示后 2.2s 的候选目标时刻对齐后，加入 H/P 代理及任务变量，特征重建的标准化 RMSE 由 **1.015 降至 0.856**；13 维脑电特征的提示方向分类平衡准确率为 52.2%。这些结果支持代理对观测特征的压缩表达，但不足以确认深部脑源或认知通路的因果关系。

关键词：脑电图 稳健黎曼伪迹拒绝 惩罚样条 神经群体动力学 认知功能代理

1 模型假设与符号说明

1.1 模型假设

1. 头皮观测采用线性叠加近似。在给定电极与参考方式下，脑电可近似表示为多个源分量的线性组合与噪声之和。问题二据此构造固定的源到电极映射；其源位置与方向属于模型设定，三电极观测不用于唯一反演五个源的活动。
2. 同一记录中的非异常试次具有可比较的协方差结构。假设明显伪迹会使局部频带能量或通道协方差偏离主要试次群体，因而可用稳健黎曼重心及中位数绝对偏差建立筛选参照。真实认知响应也可能偏离该中心，筛选阈值的影响需结合波形和敏感性分析判断。
3. 提示前短时段可作为局部基线。假设所用基线窗内没有与当前提示同步的系统性响应，以其均值校正试次偏置。条件平均时，假设目标响应与提示保持一定时间一致性，非锁时干扰可经叠加平均部分抵消；窗口内检出的正向峰仅作为 ERP 候选峰。
4. 视觉提示由亮度差及空间构型描述。左右提示采用相同呈现时长，模型输入保留边缘方向、顶点位置与三条边的相对排列。提示出现和消失共同驱动 ON/OFF 分支，提示消失后的活动按状态方程衰减。问题二主模型仅模拟提示事件。

5. 群体状态采用低维动力学近似。以简化 LGN 反馈回路及三组 Wilson–Cowan 型兴奋–抑制方程描述视觉加工，忽略单神经元的个体差异。同一拟合组、同一电极的左右条件共享动力学参数与幅度因子，使条件差异由输入及其后续计算产生。
6. 认知代理描述试次内的统计关联。用三通道 ERP 均值、 θ 与 β 频带对数功率分别构造 $V/H/P$ 代理，采用线性关系近似其在当前数据范围内的关联。代理名称表示建模时赋予的功能含义，不表示已分离出 LGN、海马或前额叶的真实源活动，也不赋予回归路径因果含义。
7. 目标时刻采用候选对齐约定。在缺少独立目标触发标记时，以提示后 2.2s 为候选目标时刻，并通过 2.0–2.4s 的偏移分析检查结果变化。应答前窗口终点取候选应答标记前 100ms，用于减轻动作相关干扰，但不假定该处理能够完全消除干扰。
8. 记录是泛化评价的基本单位。将整份 MAT 文件作为留出单元，标准化等需要数据估计的处理仅在训练记录上拟合。由于受试者身份未经核实，验证结果限于跨记录表现；问题三从原始脑电独立提取特征，其样本数及质量筛选口径与前两问分别统计。

1.2 符号说明

下表列出贯穿模型建立与结果评价的主要符号。各频带功率及原始 ERP 幅值沿用数据自身的量纲；未经校准的幅值不另标为微伏。问题二的时间常数以毫秒计，试次对齐时间通常以秒计。

表 1: 主要符号及含义

符号	含义
试次质量筛选与 ERP 提取	
i, C, T	试次索引、通道数及所用时间窗的采样点数；本文 $C = 3$
$\mathbf{X}_i$	第 i 个试次的脑电矩阵，维度为 $C \times T$
$\mathbf{X}_i^{\text{SQI}}, \mathbf{X}_i^{\text{ERP}}$	SQI 窗口信号与清洗后完成基线校正的 ERP 试次信号
$\Sigma_i, \bar{\Sigma}$	试次协方差矩阵与稳健黎曼重心
$\delta_R(\cdot, \cdot), d_i$	仿射不变黎曼距离及试次到参考重心的距离
$\mathcal{S}^{(k)}$	第 k 轮稳健重心更新所用的参考试次集合
MAD	中位数绝对偏差，用于稳健标准化
$f_i^{(j)}, u_i^{(j)}, w_j$	第 j 个检测器的特征、正异常分数与组合权重
D_i, SQI_i	综合异常分数与信号质量指数， $\text{SQI}_i = \exp(-D_i)$
τ^{gap}, τ	SQI 最大间隔阈值与最终筛选阈值
α (问题一)	B 组 SQI 阈值缩放系数，正式方案取 0.6
$\text{ERP}_{c,n}(t)$	提示条件 c 、电极 n 的条件平均 ERP
$\hat{A}_{c,n}, \hat{\mu}_{c,n}$	候选窗口内的峰值及其相对提示起点的时间
$B_{j,3}(t), c_j, \lambda$	三次 B 样条基函数、拟合系数及二阶差分惩罚参数
视觉刺激正向计算	
$X_s(x, y)$	提示图像相对共同背景的归一化亮度差， $s \in \{L, R\}$
D_t, A_t, T_t	中心–周围响应、适应状态及适应后的瞬态响应
$U_{\text{ON}}, U_{\text{OFF}}$	ON、OFF 两分支的中继输入
R, N, Q	TCR、IN、TRN 三类神经群的相对活动

续下页

表 ?? (续)

符号	含义
$\Phi_{\beta,\theta}$	输出限制在 $[0, 1]$ 内的截断逻辑激活函数
V_θ, B, H_L, H_R	Gabor 方向能量、合成方向能量及左右三角构型响应图
d_L, d_R	左、右三角模板偏好驱动
$E_{p,c}, I_{p,c}$	第 p 组、第 c 通道的兴奋与抑制状态
τ_s, g_i, τ_a	皮层时间常数、抑制增益及 LGN 适应时间常数
$h_\tau, P_{p,c}$	因果突触响应核及滤波后的群体源代理分量
$\mathbf{q}(t)$	五维功能源代理向量
$\mathbf{G}, \mathbf{R}, \mathbf{G}_{\text{eff}}$	基础映射、固定观测模式变换及有效输出映射, $\mathbf{G}_{\text{eff}} = \mathbf{R}\mathbf{G}$
α (问题二)	按拟合组与电极估计、左右条件共用的有符号幅度因子
$\mathbf{z}$	三电极在三个固定时窗内的均值组成的九维特征向量
认知功能代理与评价指标	
$t_{\text{cue}}, t_{\text{target}}, t_{\text{act}}$	提示时刻、候选目标时刻及候选应答时刻
$E_{i,c}, L_{i,c,b}$	试次 i 、通道 c 的 ERP 均值与频带 b 的对数功率
$A_{i,\alpha}$	额区 α 频带功率不对称指标, 等于 F4 与 F3 对数功率之差
$\mathbf{f}_i$	用于提示方向解码的十三维脑电特征向量
V_i, H_i, P_i	视觉前馈、记忆匹配、认知控制类功能代理
u_i, T_i^*	左右提示编码及中心化任务变量
$\alpha_V, \alpha_H, \alpha_P$	三个代理关联方程的截距
$\beta_V, \beta_{HV}, \beta_{HT}$	提示到 V 、 V 到 H 、任务到 H 的回归系数
$\beta_{PH}, \beta_{PV}, \beta_{PT}$	H 、 V 及任务变量到 P 的回归系数
$R^2, \text{RMSE}, \text{NRMSE}$	拟合优度、均方根误差及按目标曲线均方根归一化的误差
BA, AUC	平衡准确率与 ROC 曲线下面积

不同章节的同形符号按所在模型区分: 问题一与问题二的 α 分别为阈值系数和幅度因子, 频带下标 α 表示阿尔法频带; T 为采样点数, T_t 为瞬态响应, T_i^* 为任务变量; H_L, H_R 为图像构型响应, H_i 为试次级功能代理。 $E_{p,c}$ 表示群体状态, $E_{i,c}$ 表示 ERP 均值。未加粗的 R 为 LGN 中继群活动, $\mathbf{R}$ 为观测模式变换矩阵。

2 模型评价与推广

2.1 模型优点

本文围绕三通道数据组织计算, 方法选择与观测条件相适应。伪迹筛选将协方差结构与分频段异常结合, 对位置提示和形状提示任务采用不同检测器配置; 阈值敏感性和等样本量抽样同时检查清洗强度及样本组成变化, 便于发现局部峰值受筛选影响的情况。P-spline 则给出完整 ERP 轨迹, 保留了候选峰以外的响应信息。

正向模型保留边缘方向及空间位置, 可逐级追踪视觉输入经群体活动形成电极差异的过程, 并以空间消融检查模型内部的差异来源。认知模型直接从实测脑电构造功能代理, 便于对照各频段的贡献。评价中分别报告总体波形、左右差异波和记录留出分类结果, 避免仅凭平均曲线吻合就推断单试次判别有效。

2.2 模型不足

三电极对五源的映射不唯一，固定源坐标和观测模式变换限制了生理解释。总体 ERP 虽取得较高样本内相关性，左右差异波仍未复现；六组参数拟合仅一组报告收敛，部分候选参数接近搜索边界。第一问的局部峰值对筛选阈值敏感，较强清洗可能改变条件样本组成。第三问代理降低了特征重建误差，但分类及应答侧预测未呈现稳定增益；其重建结果也不能作为独立脑源的证据。候选目标时刻、有限记录数及尚待核实的行为标记进一步限制了结论范围。

2.3 模型推广

该流程可作为同类少通道视觉实验的分析起点：更换任务时，重新确定事件窗、检测频段与阈值，并检查保留样本的构成。正向模型可通过更换形状模板研究其他视觉图形，再以独立记录检验所预测的差异。若补充顶枕区电极、个体头模型及可靠目标触发标记，可进一步约束映射并检验认知代理与行为的关系。用于脑机接口或疾病研究前，还需扩大受试者样本、加入相应标签并开展独立验证，现有结果尚不足以支持临床诊断。

A 问题一：稳健黎曼伪迹筛选

```
1 # 第一问核心算法：黎曼几何质量指数计算与试次筛查
2 # 以下函数摘自第八步正式源程序，省略公共导入和参数定义，保留算法逻辑
3
4 # 对称正定矩阵进行谱分解，用于计算矩阵对数、平方根等矩阵函数
5 def matrix_function(matrix, function, positive=False):
6     values, vectors = eigh((matrix + matrix.T) / 2)
7     if positive:
8         values = np.maximum(values, 1e-12)
9     return (vectors * function(values)) @ vectors.T
10
11
12 # 计算单个试次的通道协方差矩阵，并加入微小正则项
13 def covariance_matrix(signals):
14     centered = signals - signals.mean(axis=1, keepdims=True)
15     covariance = centered @ centered.T / (centered.shape[1] - 1)
16     return covariance + 1e-6 * np.eye(centered.shape[0])
17
18
19 # 计算两个协方差矩阵之间的黎曼距离
20 def riemann_distance(first, second):
21     inverse_root = matrix_function(first, lambda x: 1 / np.sqrt(x), positive=True)
22     relative = inverse_root @ second @ inverse_root
23     eigenvalues = eigh((relative + relative.T) / 2, eigvals_only=True)
24     return np.linalg.norm(np.log(np.maximum(eigenvalues, 1e-12)))
25
26
27 # 迭代计算协方差矩阵组的黎曼均值
28 def riemann_mean(covariances):
29     log_mean = np.mean(
30         [matrix_function(cov, np.log, positive=True) for cov in covariances], axis=0
31     )
32     center = matrix_function(log_mean, np.exp)
33     for _ in range(30):
34         root = matrix_function(center, np.sqrt, positive=True)
35         inverse_root = matrix_function(center, lambda x: 1 / np.sqrt(x), positive=True)
36         delta = np.mean(
37             [matrix_function(inverse_root @ cov @ inverse_root, np.log, positive=True)
38              for cov in covariances],
39             axis=0,
40         )
41         if np.linalg.norm(delta, "fro") < 1e-8:
42             break
43         center = root @ matrix_function(delta, np.exp) @ root
44         center = (center + center.T) / 2
45     return center
46
47
48 # 依据黎曼距离和稳健标准分，迭代识别协方差矩阵内点
49 def robust_riemann_center(covariances):
50     keep = np.ones(len(covariances), dtype=bool)
51     for _ in range(4):
52         center = riemann_mean(covariances[keep])
```

```

53     distances = np.array([riemann_distance(cov, center) for cov in covariances])
54     reference = distances[keep]
55     median = np.median(reference)
56     mad = np.median(np.abs(reference - median))
57     z_score = (distances - median) / (1.4826 * mad + 1e-12)
58     updated = z_score <= 3
59     if np.array_equal(updated, keep):
60         break
61     keep = updated
62     return riemann_mean(covariances[keep]), keep
63
64
65 # 对指定频带进行四阶零相位带通滤波
66 def bandpass(data, sample_rate, low_hz, high_hz):
67     sos = butter(4, [low_hz, high_hz], btype="bandpass", fs=sample_rate, output="sos")
68     return sosfiltfilt(sos, data, axis=-1)
69
70
71 # 以稳健内点为参照，将异常特征转换为非负稳健标准分
72 def positive_robust_z(feature, inliers):
73     reference = feature[inliers]
74     median = np.median(reference)
75     mad = np.median(np.abs(reference - median))
76     z_score = (feature - median) / (1.4826 * mad + 1e-12)
77     return np.maximum(0, np.clip(z_score, -3, 6))
78
79
80 # 在质量指数排序曲线的前 40% 范围内，以最大相邻跳变确定拐点
81 def sqi_knee_threshold(sqi):
82     ordered = np.sort(sqi)
83     if len(ordered) < 2:
84         raise ValueError("At least two non-clipped trials are required to estimate SQI threshold")
85     search_end = min(len(ordered) - 1, max(3, int(len(ordered) * 0.4)))
86     return ordered[np.argmax(np.diff(ordered[:search_end + 1])) + 1]
87
88
89 # 按组别配置当前项目实际使用的频带、通道和异常度量
90 def feature_specs(is_group_a):
91     if is_group_a:
92         return [
93             ("RP1_F3F4_1_7_Frobenius", "low", (1, 2), "frobenius"),
94             ("RP2_Fz_1_7_Variance", "low", (0,), "variance"),
95             ("RP3_All_16_24_Trace", "high", (0, 1, 2), "trace"),
96             ("RP4_All_1_24_Riemann", "full", (0, 1, 2), "riemann"),
97         ]
98     return [
99         ("RP1_All_1_7_Riemann", "low", (0, 1, 2), "riemann"),
100         ("RP2_F3F4_16_24_Trace", "high", (1, 2), "trace"),
101         ("RP3_All_1_24_Riemann", "full", (0, 1, 2), "riemann"),
102     ]
103
104
105 # 计算各试次的频带异常特征、质量指数和组别阈值

```

```

106 def calculate_sqi(data, sample_rate, time, valid_indices, is_group_a):
107     epoch = (time >= EPOCH_START) & (time <= EPOCH_END)
108     if not epoch.any():
109         raise ValueError(f"No samples in the analysis window {EPOCH_START} - {EPOCH_END} s")
110
111     bands = {
112         "low": bandpass(data, sample_rate, 1, 7),
113         "high": bandpass(data, sample_rate, 16, 24),
114         "full": data,
115     }
116     feature_names, feature_values, normalized = [], [], []
117
118     for feature_name, band, channels, metric in feature_specs(is_group_a):
119         channel_indices = np.asarray(channels)
120         covariances = np.stack([
121             covariance_matrix(bands[band][trial, channel_indices][:, epoch])
122             for trial in valid_indices
123         ])
124         center, inliers = robust_riemann_center(covariances)
125
126         if metric == "frobenius":
127             values = np.linalg.norm(covariances, axis=(1, 2))
128         elif metric == "variance":
129             values = covariances[:, 0, 0]
130         elif metric == "trace":
131             values = np.trace(covariances, axis1=1, axis2=2)
132         else:
133             values = np.array([riemann_distance(cov, center) for cov in covariances])
134
135         feature_names.append(feature_name)
136         feature_values.append(values)
137         normalized.append(positive_robust_z(values, inliers))
138
139     features = np.column_stack(feature_values)
140     anomaly_score = np.mean(normalized, axis=0)
141     sqi_valid = np.exp(-anomaly_score)
142     knee = sqi_knee_threshold(sqi_valid)
143     threshold = knee if is_group_a else max(0.05, 0.6 * knee)
144     return feature_names, features, sqi_valid, threshold
145
146
147 # 削顶试次不参与质量指数计算；两类异常标记取并集作为最终剔除结果
148 valid_indices = np.flatnonzero(~drop_values.astype(bool))
149 feature_names, features, sqi_valid, threshold = calculate_sqi(
150     eeg, sample_rate, time, valid_indices, dataset_name.startswith("VisualCogA")
151 )
152 riemann_drop = np.zeros(len(trials), dtype=bool)
153 riemann_drop[valid_indices] = sqi_valid < threshold
154 final_drop = drop_values.astype(bool) | riemann_drop

```

B 问题二：视觉刺激正向脑电模型

```
1 # 问题二核心算法：视觉前端提供 B、shape、dir_left、dir_right 特征。
2 # 以下保留皮层群体动力学、功能源映射和头皮电极正向投影。
3
4 W = {"w_ee": 1.4, "w_ei": 1.1, "w_ie": 1.0, "w_ii": 0.8,
5      "g_p": 2.2, "g_q": 1.2, "tau_e": 18.0, "tau_i": 10.0,
6      "tau_i_ratio": 0.55, "delay_early": 8.0, "delay_shape": 33.0,
7      "feed_e": 0.9, "feed_i": 0.45}
8 SIGMA, RADIUS = 0.33, 0.09
9 ELECTRODES = np.array([[ -35, 55, 62], [0, 66, 61], [35, 55, 62]], float)
10 REFERENCES = np.array([[ -70, -50, 5], [70, -50, 5]], float)
11 SOURCES = np.array([[ -10, -58, 45], [10, -58, 45], [-36, -25, 50],
12                    [36, -25, 50], [0, -34, 56]], float)
13
14
15 def activation(x, beta, threshold):
16     z = np.clip(beta * (np.asarray(x) - threshold), -60, 60)
17     f0 = 1 / (1 + np.exp(beta * threshold))
18     return np.clip((1 / (1 + np.exp(-z)) - f0) / (1 - f0), 0, 1)
19
20
21 def project_features(frontend, scales=None):
22     # 空间图按左右视野平均；两个模板偏好通道保持分离。
23     def halves(x):
24         mid = x.shape[1] // 2
25         return np.stack([x[:, :mid].mean((0, 1)), x[:, mid:].mean((0, 1))]) / np.sqrt(2)
26     drives = np.stack([halves(frontend["B"]), halves(frontend["shape"]),
27                       [frontend["dir_left"], frontend["dir_right"]]]).astype(np.float32)
28     return drives if scales is None else drives / np.asarray(scales)[:, :, None]
29
30
31 def delay_signal(x, time_ms, delay_ms):
32     target = time_ms - delay_ms
33     rows = np.asarray(x).reshape(-1, len(time_ms))
34     return np.stack([np.interp(target, time_ms, row, left=0, right=0)
35                     for row in rows]).reshape(x.shape)
36
37
38 def history_at(x, step, delay_samples):
39     # 只从已计算状态取样，并对分数延迟作线性插值。
40     whole = int(np.floor(delay_samples))
41     frac, index = delay_samples - whole, step - whole
42     if index < 0 or (index == 0 and frac > 1e-12):
43         return np.zeros(x.shape[:-1], dtype=x.dtype)
44     current = x[..., index]
45     previous = x[..., index - 1] if index else np.zeros_like(current)
46     return (1 - frac) * current + frac * previous
47
48
49 def wilson_cowan(drives, time_ms, tau_s=40.0, g_i=1.0):
50     # 三群体分别表示早期对比、空间构型和三角模板偏好。
51     dt = np.median(np.diff(time_ms))
52     drive = np.empty_like(drives, dtype=np.float32)
```

```

53 drive[0] = delay_signal(drives[0], time_ms, W["delay_early"])
54 drive[1:] = delay_signal(drives[1:], time_ms, W["delay_shape"])
55 e, inh = np.zeros_like(drive), np.zeros_like(drive)
56 tau_e = np.array([W["tau_e"], tau_s, tau_s])[:, None]
57 tau_i = tau_e * np.array([W["tau_i"] / W["tau_e"], W["tau_i_ratio"], W["tau_i_ratio"]])[:,
    None]
58 for t in range(len(time_ms) - 1):
59     e_in = W["w_ee"] * e[:, :, t] - g_i * W["w_ei"] * inh[:, :, t] + W["g_p"] * drive[:, :,
    t]
60     i_in = W["w_ie"] * e[:, :, t] - W["w_ii"] * inh[:, :, t] + W["g_q"] * drive[:, :, t]
61     ef, inf = history_at(e[0], t, W["delay_shape"] / dt), history_at(inh[0], t, W["
    delay_shape"] / dt)
62     e_in[1] += W["feed_e"] * ef
63     e_in[2] += W["feed_e"] * ef.mean()
64     i_in[1] += W["feed_i"] * inf
65     i_in[2] += W["feed_i"] * inf.mean()
66     fe, fi = activation(e_in, 5, 0.35), activation(i_in, 5, 0.35)
67     e[:, :, t + 1] = e[:, :, t] + dt * (-e[:, :, t] + (1 - e[:, :, t]) * fe) / tau_e
68     inh[:, :, t + 1] = inh[:, :, t] + dt * (-inh[:, :, t] + (1 - inh[:, :, t]) * fi) / tau_i
69 return e, inh
70
71
72 def source_currents(e, inh, time_ms):
73     # 兴奋和抑制活动经突触核滤波，再映射为四个半球源和一个中线对手源。
74     dt = np.median(np.diff(time_ms))
75     def kernel(tau):
76         t = np.arange(len(time_ms)) * dt
77         h = t * np.exp(-t / tau) / tau**2
78         return h / (h.sum() * dt)
79     pe = fftconvolve(e, kernel(10)[None, None, :], mode="full", axes=-1)[..., :len(time_ms)] *
    dt
80     pi = fftconvolve(inh, kernel(20)[None, None, :], mode="full", axes=-1)[..., :len(time_ms)] *
    dt
81     early, shape, preference = pe - pi
82     return np.stack([early[1], early[0], shape[1], shape[0],
83         preference[0] - preference[1]])
84
85
86 def leadfield():
87     # 点偶极均匀导体近似，电极位于头表并使用双侧乳突参考。
88     surface = lambda p: RADIUS * p / np.linalg.norm(p, axis=1, keepdims=True)
89     electrodes, references, sources = surface(ELECTRODES), surface(REFERENCES), SOURCES / 1000
90     normals = sources / np.linalg.norm(sources, axis=1, keepdims=True)
91     def potential(points):
92         delta = points[:, None] - sources[None]
93         distance = np.linalg.norm(delta, axis=-1)
94         return np.einsum("sc,esc->es", normals, delta) / (4 * np.pi * SIGMA * distance**3)
95     return potential(electrodes) - potential(references).mean(axis=0, keepdims=True)
96
97
98 def simulate_q2(frontend, time_ms, tau_s=40.0, g_i=1.0, scales=None):
99     e, inh = wilson_cowan(project_features(frontend, scales), time_ms, tau_s, g_i)
100     sources = source_currents(e, inh, time_ms)
101     return leadfield() @ sources

```

C 问题三：认知状态与留记录验证

```
1
2
3 # Q3 动态认知状态模型的名义参数
4 PARAMS = {
5     "tau_visual": 0.060,
6     "tau_memory": 1.0,
7     "tau_control": 0.250,
8     "cue_storage_gain": 0.8,
9     "target_memory_gain": 0.8,
10    "memory_to_control_gain": 0.25,
11    "conflict_gain": 0.8,
12    "topdown_control_gain": 0.10,
13 }
14
15 def make_event_gates(time_s, cue_onset_s=0.0, cue_duration_s=0.203125,
16                      target_onset_s=2.2, target_duration_s=0.2):
17     """根据事件时刻生成提示与目标的阶跃门控信号。"""
18     time_s = np.asarray(time_s, dtype=float).reshape(-1)
19     cue = ((time_s >= cue_onset_s) &
20            (time_s < cue_onset_s + cue_duration_s)).astype(float)
21     target = np.zeros_like(cue)
22     if target_onset_s is not None:
23         if target_duration_s is None:
24             target = (time_s >= target_onset_s).astype(float)
25         else:
26             target = ((time_s >= target_onset_s) &
27                      (time_s < target_onset_s + target_duration_s)).astype(float)
28     return cue, target
29
30 def integrate_macro_states(visual_drive, cue_gate, target_gate, match_evidence,
31                             dt_s, params=PARAMS):
32     """用指数欧拉递推计算视觉、记忆和控制状态。"""
33     drive = np.asarray(visual_drive, dtype=float).reshape(-1)
34     cue = np.asarray(cue_gate, dtype=float).reshape(-1)
35     target = np.asarray(target_gate, dtype=float).reshape(-1)
36     rho_v, rho_h, rho_p = [
37         np.exp(-dt_s / params[key])
38         for key in ("tau_visual", "tau_memory", "tau_control")
39     ]
40     visual = np.zeros_like(drive)
41     memory = np.zeros_like(drive)
42     control = np.zeros_like(drive)
43     visual[0] = drive[0]
44     for k in range(1, drive.size):
45         visual[k] = rho_v * visual[k - 1] + (1 - rho_v) * drive[k]
46         memory_input = (
47             params["cue_storage_gain"] * visual[k] * cue[k]
48             + params["target_memory_gain"] * visual[k] * target[k]
49             * match_evidence
50         )
51         memory[k] = rho_h * memory[k - 1] + (1 - rho_h) * memory_input
52         conflict = target[k] * (1 - match_evidence) / 2
```

```

53     control_input = (
54         params["memory_to_control_gain"] * abs(memory[k - 1]) * target[k]
55         + params["conflict_gain"] * conflict
56     )
57     control[k] = rho_p * control[k - 1] + (1 - rho_p) * control_input
58     return {"visual": visual, "memory": memory, "control": control}
59
60 def observe_macro_model(q2_sensor, memory, control,
61                         memory_loading, control_loading, params=PARAMS):
62     """将 Q2 视觉响应及记忆、控制状态映射到 F3、Fz、F4。"""
63     q2 = np.asarray(q2_sensor, dtype=float)
64     h = np.asarray(memory, dtype=float).reshape(-1)
65     p = np.asarray(control, dtype=float).reshape(-1)
66     l_h = np.asarray(memory_loading, dtype=float).reshape(-1)
67     l_p = np.asarray(control_loading, dtype=float).reshape(-1)
68     return (q2 * (1 + params["topdown_control_gain"] * p)[None, :]
69             + l_h[:, None] * h[None, :]
70             + l_p[:, None] * p[None, :])
71
72 def _features_for_channel(q2_channel, memory, control, interaction):
73     """按通道构造视觉、记忆、控制及视觉反馈特征。"""
74     return np.column_stack((
75         q2_channel.reshape(-1),
76         memory.reshape(-1),
77         control.reshape(-1),
78         interaction.reshape(-1),
79     ))
80
81 def fit_sensor_mapping(observed, q2_sensor, memory, control,
82                       q2_control_interaction, ridge_alpha=1.0):
83     """仅用训练记录拟合各 EEG 通道的岭回归观测映射。"""
84     y = np.asarray(observed, dtype=float)
85     q2 = np.asarray(q2_sensor, dtype=float)
86     h = np.asarray(memory, dtype=float)
87     p = np.asarray(control, dtype=float)
88     interaction = np.asarray(q2_control_interaction, dtype=float)
89     fits = []
90     for channel in range(3):
91         x = _features_for_channel(
92             q2[:, channel], h, p, interaction[:, channel]
93         )
94         target = y[:, channel].reshape(-1)
95         mean_x = x.mean(axis=0)
96         scale_x = x.std(axis=0)
97         scale_x[scale_x < 1e-12] = 1.0
98         z = (x - mean_x) / scale_x
99         mean_y = target.mean()
100         beta = np.linalg.solve(
101             z.T @ z + ridge_alpha * np.eye(z.shape[1]),
102             z.T @ (target - mean_y),
103         )
104         fits.append((mean_x, scale_x, mean_y, beta))
105     return fits
106

```

```

107 def predict_sensor_mapping(fits, q2_sensor, memory, control,
108                             q2_control_interaction):
109     """将训练阶段估计的映射应用于留出记录。"""
110     q2 = np.asarray(q2_sensor, dtype=float)
111     h = np.asarray(memory, dtype=float)
112     p = np.asarray(control, dtype=float)
113     interaction = np.asarray(q2_control_interaction, dtype=float)
114     prediction = np.empty_like(q2)
115     for channel, (mean_x, scale_x, mean_y, beta) in enumerate(fits):
116         x = _features_for_channel(
117             q2[:, channel], h, p, interaction[:, channel]
118         )
119         prediction[:, channel] = (
120             mean_y + ((x - mean_x) / scale_x) @ beta
121         ).reshape(q2.shape[0], q2.shape[2])
122     return prediction
123
124 def score_prediction(observed, predicted, time_s, window):
125     """计算指定时间窗内的 RMSE、标准化 RMSE 与相关系数。"""
126     mask = (time_s >= window[0]) & (time_s < window[1])
127     actual = np.asarray(observed)[..., mask].reshape(-1)
128     estimate = np.asarray(predicted)[..., mask].reshape(-1)
129     rmse = np.sqrt(np.mean((actual - estimate) ** 2))
130     scale = np.std(actual)
131     nrmse = rmse / scale if scale > 1e-12 else np.nan
132     corr = (np.corrcoef(actual, estimate)[0, 1]
133             if np.std(actual) > 1e-12 and np.std(estimate) > 1e-12
134             else np.nan)
135     return {"RMSE": rmse, "NRMSE": nrmse, "相关系数": corr}
136
137 def main():
138     """依次执行事件审计、Q2场景模拟、Q3留记录验证与结果汇总。"""
139     root = str(Path(__file__).resolve().parents[3])
140     if root not in sys.path: sys.path.insert(0, root)
141     event_audit = importlib.import_module("src.C.q3.09_event_time_semantics")
142     validation = importlib.import_module("src.C.q3.dynamic_validation")
143     event_audit.main()
144     return validation.run_validation()
145
146 if __name__ == "__main__":
147     main()

```